3장 파이썬 최고의 라이브러리 : 넘파이

학습목표

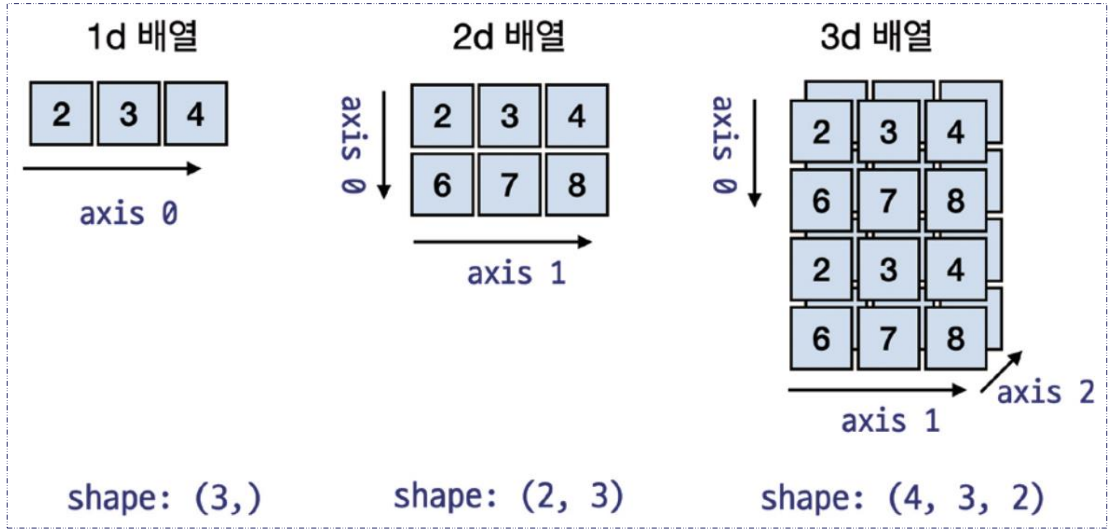
- 넘파이의 다차원 배열과 리스트의 차이를 이해한다.
- 다차원 배열의 장점과 특징을 이해한다.
- 넘파이의 브로드캐스팅과 벡터화 연산이 빠른 속도를 내는 이유를 이해한다.
- 넘파이의 강력한 함수에 대하여 익히고 응용 능력을 익힌다.

8. 주차별 강의계획(Weekly Schedule)

주 (Week)	강의주제 (Lecture subject)	강의 세부내용 (Detailed contents)
1	- 빅데이터와 인공지능 - 빅데이터 Python 환경 구성	- Git, Python, Visual Studio Code, Anaconda Jupyter notebook, Google Colab
2	- 수치 데이터 처리 : 다차원 배열	- Python 프로그래밍 기초 - Numpy
3	- 통계 데이터의 분석 : 시각화	- matplotlib, seaborn, plotly
4	- 웹크롤링에 의한 데이터 수집과 저장 (교재에 포함 안됨)	- 텍스트파일 : txt, csv, json, PDF, HWP - 공공데이터포탈 API - Naver API - Google Pytrands - BeautifulSoup, Selenium
5	- 텍스트 데이터의 탐색적 분석 : 시각화 (교재에 포함 안됨)	- 워드 클라우딩
6	- 시계열 데이터의 데이터프레임 분석 및 시각화	- Pandas
7	- 데이터 전처리	- 결측 데이터, 중복 데이터, 이상치 처리하기 - 병합 데이터 다루기
8	중간고사	- 필기시험

3.1 파이썬 리스트와 넘파이(Numpy)

- 넘파이의 **다차원 배열** ndarray 은 **동일한 자료형을** 가진 데이터를 **연속으로 저장**한다. 각 데이터 항목에 필요한 저장 공간이 일정하므로, 몇 번째 위치에 있는 항목이든 그 순서만 안다면 계산으로 바로 접근할 수 있다. 다차원 배열의 각 요소는 **인덱스** 정수로 참조된다.
- **리스트는 여러 자료형의 값**들을 **저장하는 구조**이지만 리스트 간의 연산 기능이 부족, **연산 속도도 빠르지 않다.**



- 넘파이 배열의 **차원**은 **축**axis이라고도 함. 넘파이의 1차원, 2차원, 3차원 배열 등은 축을 이용하여 그 **형상**shape을 표현. 이 형상의 의미는 **튜플 형식으로 표현**되는 다차원 **배열 내부의 각 축이 가지는 최대 원소의 수**를 나타낸다. 형상이 (3,)은 축이 하나인 1차원 배열, (2, 3)은 축이 2개인 2차원 배열. (4, 3, 2)은 축이 3개인 3차원 배열.

3.2 다차원 배열 객체 생성과 속성들

- 넘파이는 성능이 우수한 **ndarray 다차원 배열 객체**를 제공한다. 넘파이의 ndarray는 **array()** 함수에 파이썬 **리스트를 넣으면 넘파이 배열**을 생성하여 반환한다.
- 다차원 배열의 속성을 이용하여 **프로그램의 오류 또는 배열의 상세한 정보를 조회할 수 있다.**

속 성	설 명
ndim	배열 축 혹은 차원의 개수를 나타냄.
shape	배열의 형상을 기술하며 (m, n) 형식의 튜플 형으로 나타냄.
size	배열 원소의 개수를 말하며 (m, n) 형상 배열의 size는 $m \cdot n$ 임.
dtype	배열 원소의 자료형을 기술함.

```
a = np.array([2, 3, 4])        # 넘파이 ndarray 객체의 생성

# a 객체의 형상(shape), 차원, 요소의 자료형, 요소의 크기(byte), 요소의 수
a.shape, a.ndim, a.dtype, a.itemsize, a.size
```

```
((3,), 1, dtype('int64'), 8, 3)
```

```
b = np.array([[1, 2, 3], [4, 5, 6]])  # 넘파이 ndarray 객체의 b 생성
b.shape     # 2행 3열의 배열
```

```
(2, 3)
```

3.3 넘파이 다차원 배열 간 사칙 연산

- 다차원 배열 ndarray 객체 간 사칙연산은, 우선 **두 배열의 형상이 일치**할 때 **원소별로 덧셈, 뺄셈, 곱셈 나눗셈이 이루어진다**. 하지만 이 연산자들이 하는 일은 리스트와는 다르다.
- 넘파이는 **대용량의 배열과 행렬 연산을 수행하는 고차원적인 수학 연산자와 함수를** 포함하고 있으며, 넘파이는 다차원 배열의 원소에 접근하는 속도가 리스트보다 더 빠르다.
- 리스트는 배열을 여러 데이터가 나열된 것으로만 간주. 넘파이는 **배열을 선형대수의 벡터**로 가정하여 **연산을 병렬처리**하여 **배열 프로그래밍**하는 방식이다. 파이썬은 하나의 명령을 여러 데이터에 적용하여 **병렬로 연산**하는데 이것을 **벡터화 연산**vectorized operation이라 함.

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
a + b
```

```
array([11, 22, 33])
```

```
a - b
```

```
array([ 9, 18, 27])
```

```
a * b
```

```
array([10, 40, 90])
```

```
a / b
```

```
array([10., 10., 10.])
```

3.4 다차원 배열 사칙 연산과 브로드캐스팅

- 넘파이는 **배열 프로그래밍에서 배열 간 차원이 다른 경우** 사칙연산을 수행할 때는 먼저 차원을 일치시킨다. 넘파이는 a * 10 을 수행할 때 np.array([10, 20, 30]) * np.array([10, 10, 10])와 같이 a의 차원에 맞게, **스칼라 10**을 벡터로 확장시켜주는 작업을 하는데 이를 **브로드캐스팅**broadcasting이라고 한다. 브로드캐스팅과 동시에 파이썬은 하나의 명령을 여러 데이터에 적용하여 **병렬로 벡터화 연산을 수행한다**.

```
a = np.array([10, 20, 30])
a * 10
```

```
array([100, 200, 300])
```

```
a + 10
```

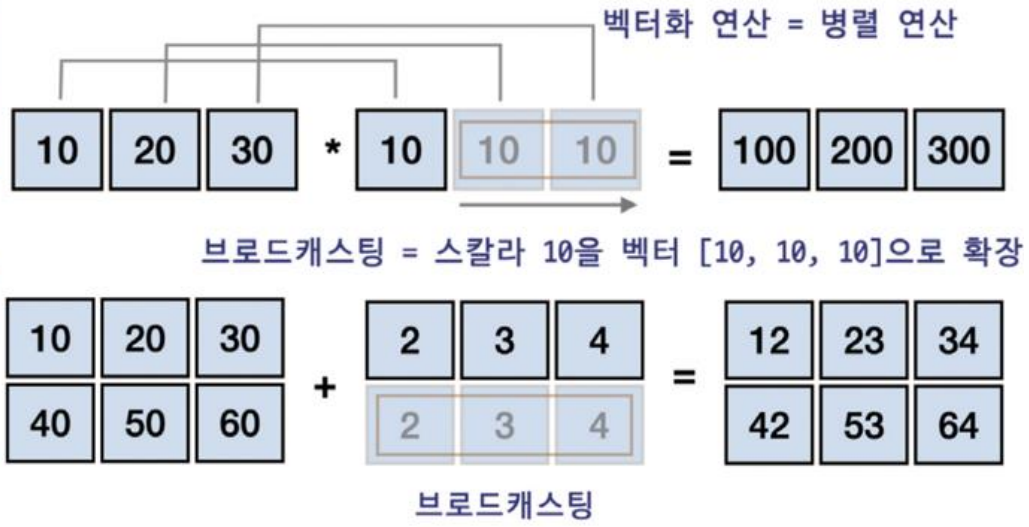
```
array([20, 30, 40])
```

```
b = np.array([[10, 20, 30],
              [40, 50, 60]])
c = np.array([2, 3, 4])
b + c
```

```
array([[12, 23, 34],
       [42, 53, 64]])
```

```
b * c
```

```
array([[ 20,  60, 120],
       [ 80, 150, 240]])
```



3.5 연속적인 값을 가지는 다차원 배열의 생성

- 넘파이는 **다차원 배열을 쉽게 생성하는 함수**도 지원하고 있다.
- zeros((n, m)) 함수는 n×m 배열을 생성하는데 그 초기값은 0으로 해준다.
- ones((n, m)) 함수를 호출하면 초기값을 1로 가지는 n×m 배열을 생성
- 초기값을 임의의 수 x로 지정하고 싶을 경우 full((n, m), x) 함수를 호출
- 행과 열의 크기가 동일한 정방행렬 중에서 대각선 성분이 1이고 나머지 성분은 0인 행렬을 **단위행렬**identity matrix, eye(n) 함수를 이용하면 n×n 크기의 단위행렬

```
np.zeros((2, 3))        # 2행 3열의 행렬 생성 시 모든 값을 0으로
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
np.ones((2, 3))         # 2행 3열의 행렬 생성 시 모든 값을 1로
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
np.full((2, 3), 100) # 2행 3열의 행렬 생성 시 모든 값을 100으로
```

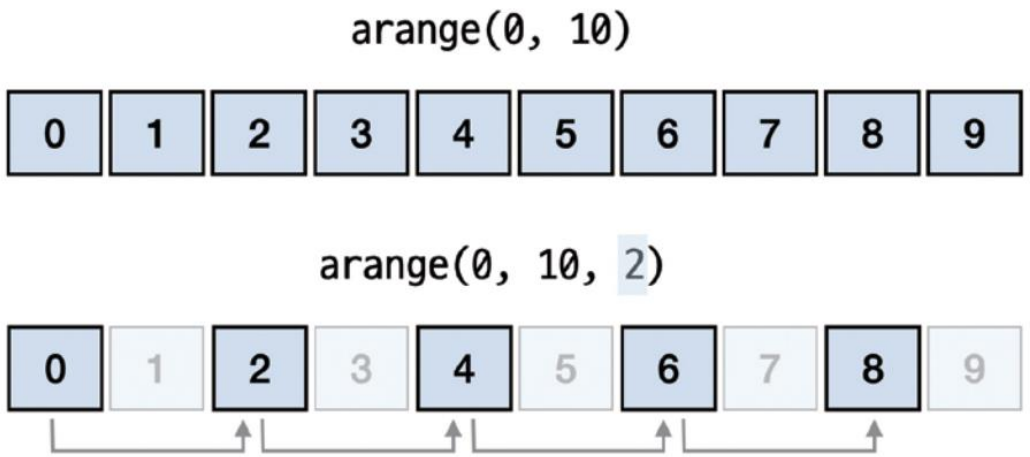
```
array([[100, 100, 100],
       [100, 100, 100]])
```

```
np.eye(3)               # 3x3 크기의 단위행렬 생성
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

3.5 연속적인 값을 가지는 다차원 배열의 생성

- 넘파이의 **숫자의 열을 생성**하는 **arange(m, n)** 함수는 리스트의 range() 함수와 유사하며, **m부터 n미만까지 연속적인 정수나 실수**를 만들어준다.
- **arange(m, n, step)**과 같이 3개의 인자를 넣어주게 되면 m부터 n까지 step만큼 건너뛰며 연속적인 정수나 실수를 생성한다. 스텝 값으로는 정수 자료형이 아니어도 가능하다.




```
np.arange(0, 10, 3)        # 3씩 증가하는 수열을 생성
array([0, 3, 6, 9])
np.arange(0.0, 1.0, 0.2)   # 0.2씩 증가하는 수열을 생성
array([0. , 0.2, 0.4, 0.6, 0.8])
```

3.5 연속적인 값을 가지는 다차원 배열의 생성

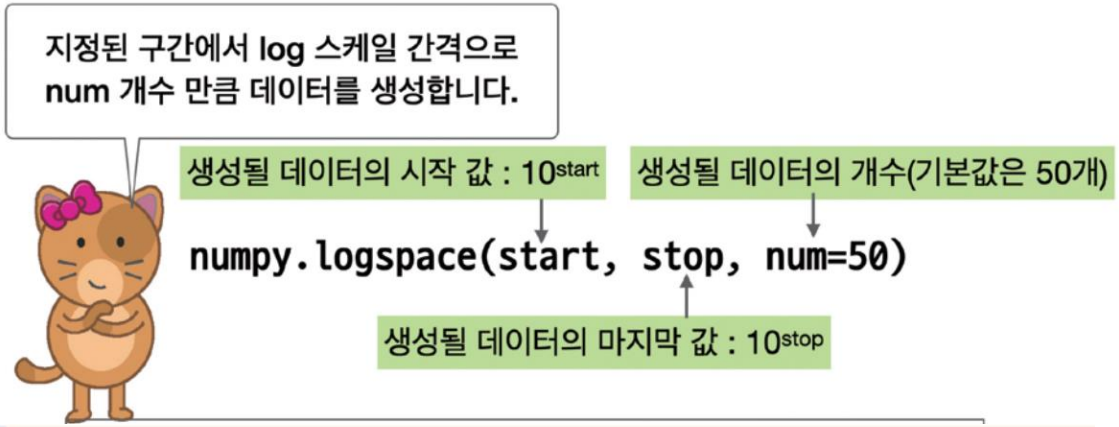
- 넘파이의 linspace()은 **지정된 구간에서 동일한 간격의 값을 지정된 횟수만큼 생성**.
- 넘파이의 logspace()함수은 **로그 스케일로** 수들을 생성.

```
np.linspace(0, 10, 5)
```

```
array([ 0. , 2.5, 5. , 7.5, 10. ])
```

```
np.linspace(0, 10, 4)
```

```
array([ 0. , 3.33333333, 6.66666667, 10. ])
```




```
a = np.logspace(0, 5, 6).round(10) # 10^0에서 10^5까지 6개의 수를
a                                  # 로그 스케일 균일한 간격으로 생성한다.
```

```
array([1.e+00, 1.e+01, 1.e+02, 1.e+03, 1.e+04, 1.e+05])
```

```
np.log10(a)     # a 배열에 로그를 취하면 0, 1, 2, 3, 4, 5가 생성됨
```

```
array([0., 1., 2., 3., 4., 5.])
```

3.6 다차원 배열의 축과 삽입

- 다차원 배열에서 **축을 이용하여 원소를 삽입**은 **insert(배열 객체, 삽입할 위치, 삽입할 값, 삽입 방향)** 메소드을 이용.
- **2차원 배열에** 대해서 **특정 값을 행과 열에 삽입하는 경우:**
  - insert(b, 1, 4, axis = 0)는 스칼라값 4을 [4, 4]와 같은 벡터값으로 **브로드캐스팅**하므로, insert(b, 1, (4, 4), axis = 0)과 동일.
  - axis = 1인 경우 **축 1의 방향(2차원 배열에서 열 방향)**의 두 번째 항목에 4를 삽입한다.

```
a = np.array([1, 3, 4])
np.insert(a, 1, 2)
```

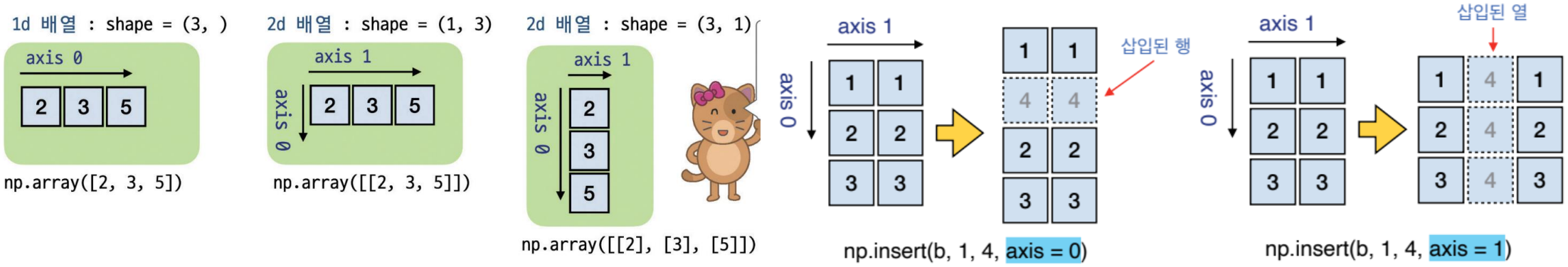
```
array([1, 2, 3, 4])
```

```
b = np.array([[1, 1], [2, 2], [3, 3]])
np.insert(b, 1, 4, axis = 0)
```

```
array([[1, 1], [4, 4], [2, 2], [3, 3]])
```

```
np.insert(b, 1, 4, axis = 1)
```

```
array([[1, 4, 1],
       [2, 4, 2],
       [3, 4, 3]])
```



3.7 다차원 배열의 축과 순서 뒤집기

- 또한, 넘파이에는 **flip() 함수**가 있어서 **지정된 축방향으로 원소들을 뒤집어 줄 수 있다.**

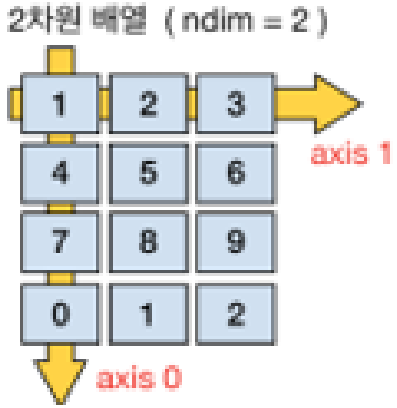
```python
# 주어진 2차원 배열을 지정된 축 방향으로  뒤집기
import numpy as np
arr = np.array([[1,2,3], ₩
                [4,5,6]])
print(arr)
print("="*100)
flip_arr_0 = np.flip(arr, axis=0)
print(flip_arr_0)
print("="*100)
flip_arr_1 = np.flip(arr, axis=1)
print(flip_arr_1)
```

```
[[1 2 3]
 [4 5 6]]
====================================================================================
[[4 5 6]
 [1 2 3]]
====================================================================================
[[3 2 1]
 [6 5 4]]
```

3.8 넘파이 배열의 인덱싱(indexing)

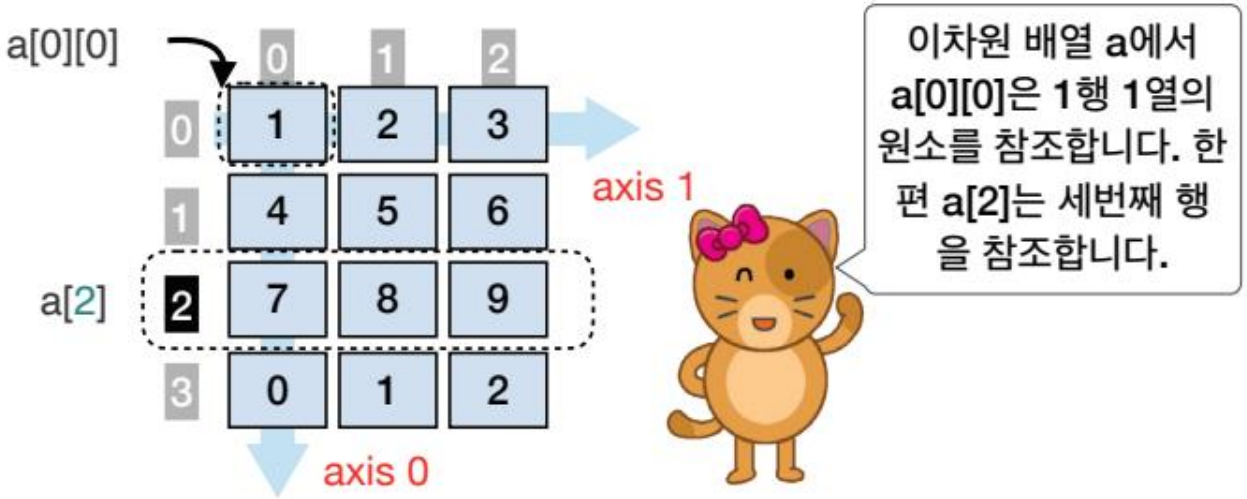
- 배열 원소는 **인덱스 정수를 사용하여 참조**할 수 있다.
- **인덱싱**은 **두 축**에 대해서 모두 가능하다.
- **첫 번째 축**으로 0, 두 번째 축으로 0을 지정하면 가장 첫 원소인 1이 표시.
- **두 번째 축에 대해 인덱싱을 하지 않으면**, 첫 번째 축에서 지정된 위치의 모든 데이터가 나타난다.

```
a = np.array( [[1, 2, 3], [4, 5, 6],
               [7, 8, 9], [0, 1, 2]] )
```



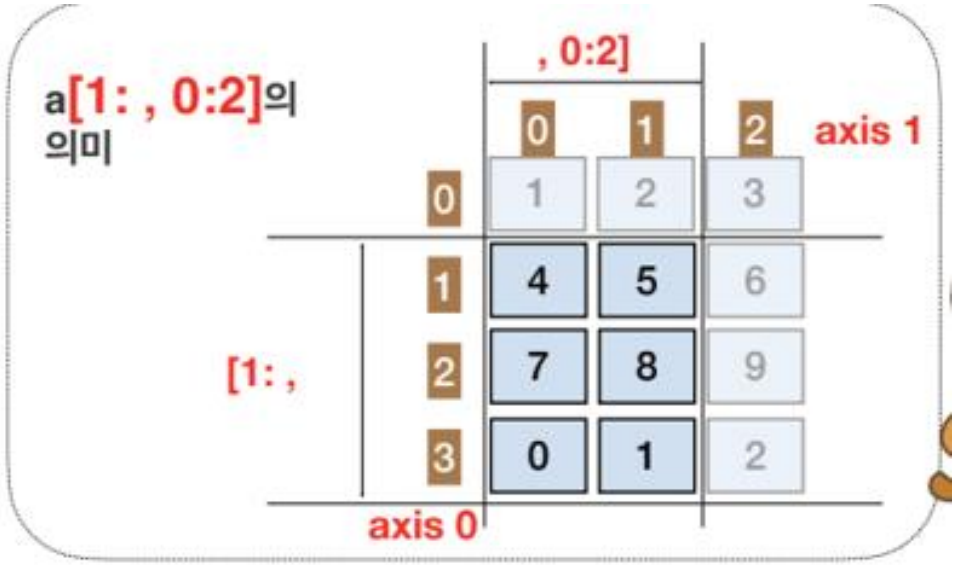

```
print(a[0][0])
print(a[2])
```

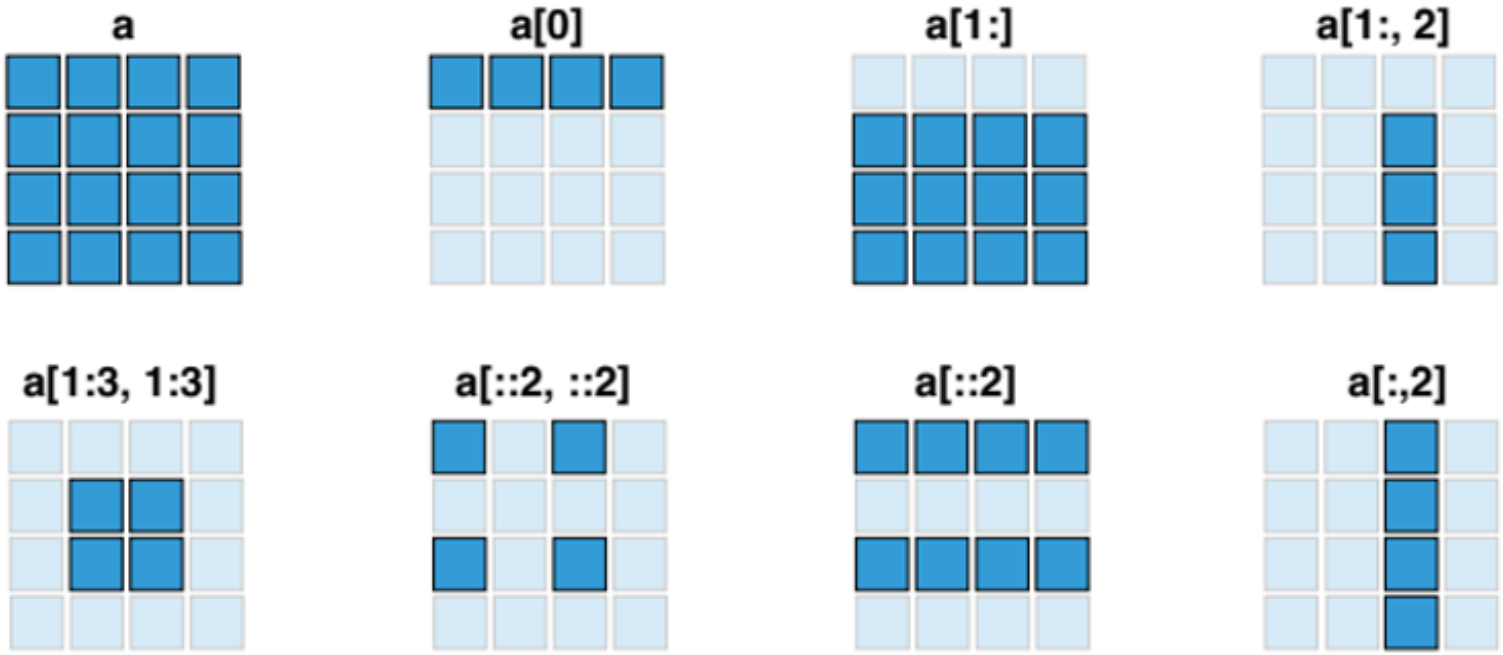
```
1
[7, 8, 9]
```



3.9 넘파이 스타일의 인덱싱을 이용하여 슬라이싱

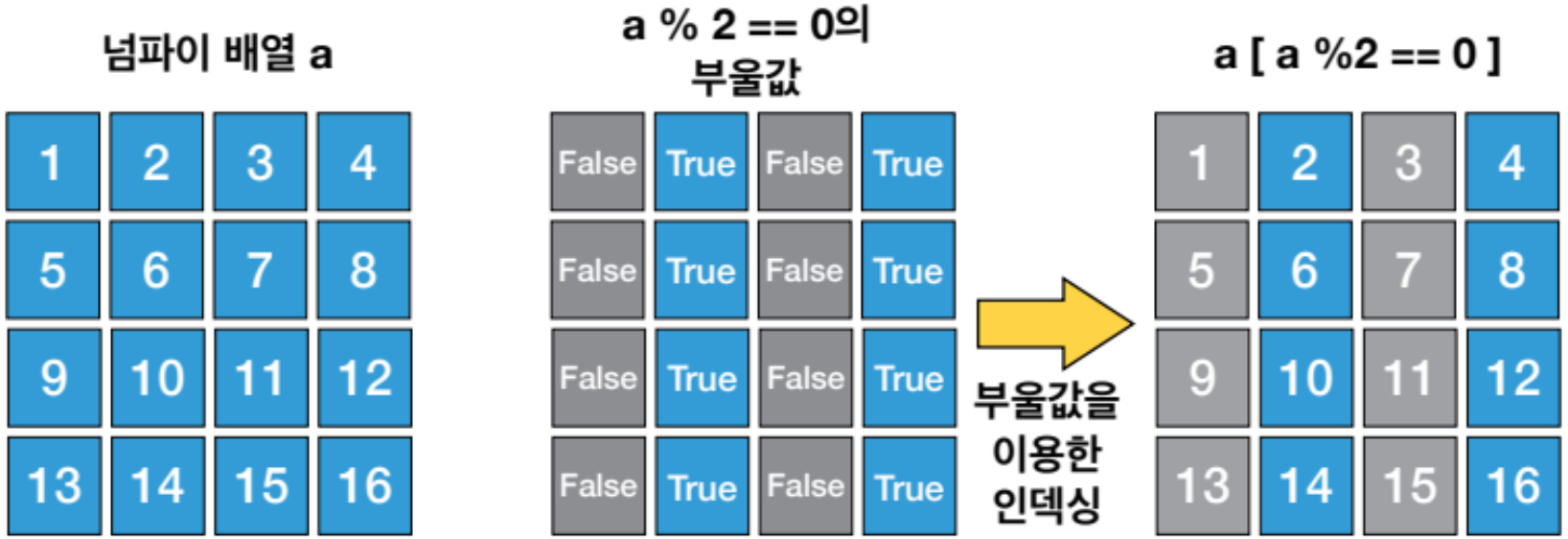
- 슬라이싱은 배열에서 **특정한 행들과 열만 추려서 작은 배열을 끄집어내는** 것.
- 넘파이의 **인덱싱을 이용**하여 **슬라이싱 범위는 쉼표와 콜론을 사용하여 각각의 축에 별도로 적용**된다. **선형대수의 행렬 관련 기능을 구현하는** 데에 중요.





3.10 넘파이 스타일의 슬라이싱과 논리 인덱싱(logical indexing)

- **논리 인덱싱**은 넘파이 **다차원배열에서 특정한 조건을 만족하는 배열 원소를 추려**내는 것.
- **넘파이 배열을 이용하여 조건식을 만들면, 불리언 배열이 생성.** 이 배열을 **인덱스로 사용하면 해당 조건에 부합하는 원소만 추출할 수 있다.** 즉, 주어진 **배열 내의 모든 원소 중에서 참인 위치의 값만 추출된다.**



3.11 다차원 배열의 평탄화, 전치행렬, 정렬

- **astype()** 메소드는 원소들의 자료형을 변환한다.
- **2차원 이상의 배열이라면** 이를 **1차원 배열로 변환할 때 flatten()** 메소드이다. 이 과정을 **평탄화**.
- 2차원 배열의 행과 열의 순서를 서로 교환하는 것을 **전치**transpose **는 .T 속성을** 이용.
- **sort() 메소드은** 원소들을 **axis 1 방향으로 오름차순** 정렬. **axis=0을 넣어주면 axis 0의 방향으로 오름차순 정렬**이 된다.

```
a = np.array([10, 20, 30])
a.max(), a.min()          # a 배열원소 중 최대값, 최소값을 출력함
```

```
(30, 10)
```

```
a.mean()                  # a의 평균을 출력함
```

```
20.0
```

```
a.astype(np.float64)      # 원소의 자료형을 float64로 변환
```

```
array([10., 20., 30.])
```

```
c = np.array([35, 24, 55, 69, 19, 99])
c.sort()        # 배열의 정렬(오름차순)
c[::-1]         # 내림차순 정렬
```

```
array([99, 69, 55, 35, 24, 19])
```

```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
d.sort()     # 배열의 정렬, 디폴트 axis는 1
d
```

```
array([[24, 35, 55],
       [ 9, 19, 69],
       [ 1,  4, 11]])
```

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.flatten()     # 넘파이 다차원 배열의 평탄화 메소드
```

```
array([1, 1, 2, 2, 3, 3])
```

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.T
```

```
array([[1, 2, 3],
       [1, 2, 3]])
```

```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]]
d.sort(axis=0)     # axis=0 방향 정렬
d
```

```
array([[ 4,  1,  9],
       [35, 19, 11],
       [69, 24, 55]])
```

3.12 배열의 형태을 바꾸는 reshape()

- **reshape()** 메소드는 **데이터의 개수는 유지한 채로 배열의 차원과 형태를 변경**한다. 이 메소드의 인자인 shape을 **튜플의 형태로 reshape(x, y)**라고 하면 **reshape((x, y))**와 동일하게 처리.

변경하여 얻고 싶은 형태를 넘겨 줌
1차원: (n,)
2차원: (n, m)
3차원: (n, m, l)

new_array = old_array.reshape(shape)

이전 배열 old_array의 형태가 (n,m)이고, 새롭게 얻고 싶은 형태 shape이 (l,k)라고 하면 $n \times m = l \times k$를 만족해야만 형태를 바꿀 수 있답니다.

```
>>> y = np.arange(12)
>>> y
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
>>> y.reshape(3, 4)
array([[ 0, 1, 2, 3],
       [ 4, 5, 6, 7],
       [ 8, 9, 10, 11]])
```

배열의 형태가 3행 4열의 모양이 되도록 만듭니다

```
>>> y.reshape(6, -1)
array([[ 0, 1],
       [ 2, 3],
       [ 4, 5],
       [ 6, 7],
       [ 8, 9],
       [10, 11]])
```

인수로 -1을 전달하면 데이터의 개수에 맞춰서 자동으로 배열의 형태가 결정

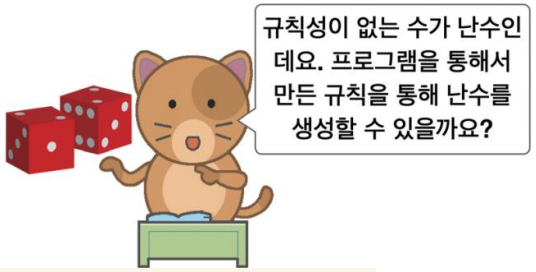
```
>>> y.reshape(7, 2)
...
    y.reshape(7, 2)
ValueError: cannot reshape array of size 12 into shape (7,2)
```

reshape()에 의해 생성될 배열의 형태가 호환되지 않을 경우 발생하는 오류

```
>>> y.flatten()    # 2차원 배열을 1차원 배열로 만들어 준다
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

flatten()은 평탄화 함수로 2차원 이상의 고차원 배열을 1차원 배열로 만들어 준다.

3.13 다양한 난수 만들기 함수를 살펴보자



- 데이터 과학에서는 **실세계의 관측을 통해서 데이터를 획득**하는 데, **많은 비용과 장시간의 관측**이 필요하다. 따라서 **실세계와 유사한 특성을 가지는 가상 데이터**를 **생성 및 분석, 시뮬레이션을 한 후에 이 모델을 실세계 데이터 분석에 적용하는** 경우가 많다.
- 넘파이 난수을 활용하면 데이터 과학을 위한 임의의 데이터를 얻을 수 있다. 난수는 **random 서브 모듈**에서 제공.

함수 이름	설명
rand()	균등분포의 난수 표본을 추출한다.
randn()	표준정규분포의 난수 표본을 추출한다.
randint()	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성한다. 이때 최대값은 포함하지 않는다.
normal()	평균과 표준 편차를 인자로 받아, 정규 분포의 난수 표본을 추출한다.
uniform()	균등 분포에서 표본을 추출한다.
shuffle()	주어진 리스트나 다차원 배열의 순서를 임의로 섞는다.
ermutation()	주어진 범위의 수를 생성하여 섞거나, 배열을 임의의 순서로 섞는다.

```
np.random.seed(42)  # 시드값이 특정되면 매번 동일한 난수의 열이 생성됨
```

```
np.random.randint(150, 191, size=10)
```

```
array([186, 157, 167, 153, 159, 173, 176, 187, 163, 179])
```

```
rnd = np.random.randn(5) * 10 + 165 # 평균값이 165, 표준편차가 10인 값 생성
rnd
```

```
array([176.07979332, 177.91821264, 165.45715947, 164.34633461,
       167.9482772 ])
```

```
rnd.round(2)          # 소수점 아래 둘째 자리까지 값으로 변환
```

```
array([176.08, 177.92, 165.46, 164.35, 167.95])
```

```
rnd.astype(int)       # 정수형 자료형으로 변환
```

```
array([176, 177, 165, 164, 167])
```

```
nums = np.random.normal(loc=165, scale=10, size=(3, 4)).round(2
nums
```

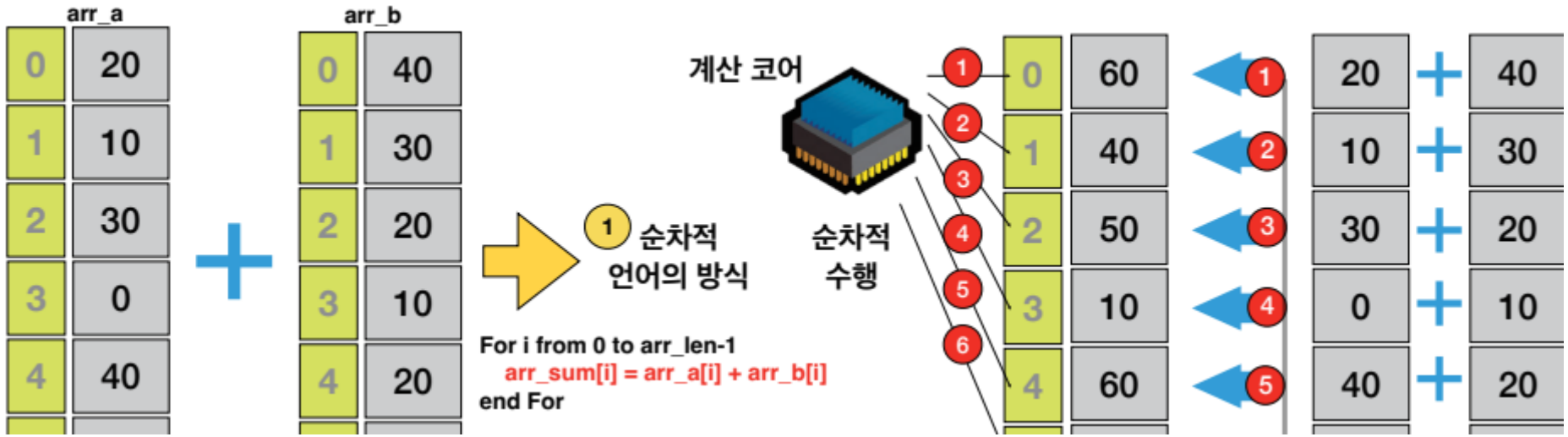
```
array([[160.32, 156.77, 164.35, 157.87],
       [174.06, 172.66, 173.26, 151.76],
       [147.48, 175.02, 170.45, 183.95]])
```

```
a = np.arange(10)       # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
a                       # a에는 순서가 뒤섞인 값이 들어있다
```

```
array([2, 8, 9, 6, 4, 0, 1, 5, 7, 3])
```

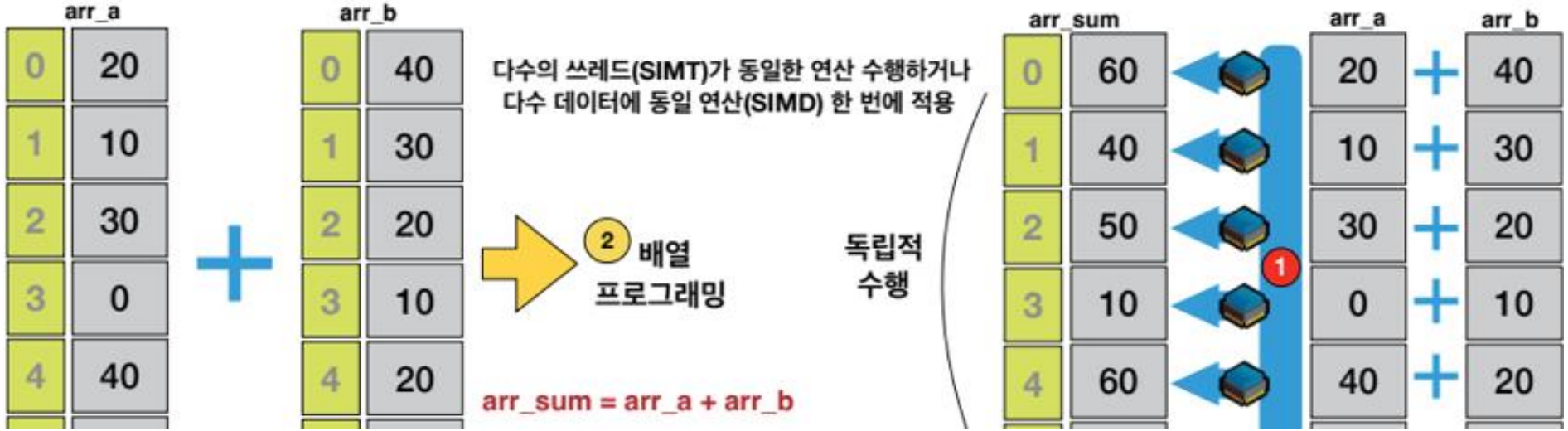
심화 넘파이는 왜 성공했나

- **배열 프로그래밍**은 **수치데이터로 구성된 배열 원소 전체에 대해 한 번에 동일 연산이 적용** 하는 방식. **연산의 표현식이 간결**. 배열 간 연산은 **벡터와 벡터의 연산를** 의미하며 이 연산을 **벡터화 연산**이라 함. 벡터화 연산은 **병렬 프로그래밍 처리**로 수행.
- 즉, 두 배열의 대응하는 원소를 서로 더해서 새로운 배열에 저장하는 일을 **For 반복문을 사용한 경우 각각의 원소들을 순차적으로 처리**



심화 넘파이는 왜 성공했나

- 병렬 프로그래밍의 **데이터 병렬성**data parallelism 방식은 벡터화 연산을 수행할 때 순서를 기다리지 않고 처리해야 하는 **데이터를 잘게 쪼개어 각각의 조각에 대해 동일한 작업을 여러 개의 코어가 동시에 독립적으로 수행하는 방식이다**. GPU는 다수의 **쓰레드**를 만들어 동일한 연산을 수행하게 하는데, 이러한 구조를 **단일 명령어 다중 쓰레드**single instruction multiple thread:SIMT 방식. CPU는 하나의 연산이 다수의 데이터에 동시에 적용되도록 하는 **단일 명령어 다중 데이터**single instruction multiple data:SIMD 방식. 넘파이을 이용하여, **프로그래머가 고성능의 데이터 병렬 처리를 할 수 있게 해준다.**



3.14 벡터화 연산 성능을 측정해 보자

- **벡터화 연산은** 하나의 명령이 여러 데이터에 동시에 적용된 방식으로 **배열 프로그래밍**이다. 큰 작업을 분할하여 다수의 프로세서에서 나누어 처리하는 전통적 병렬 처리와 달리 **하나의 프로세서 내에서 병렬적 데이터 처리**가 이루어진다. 현재 대다수의 **CPU가 SIMD 방식을 지원하는데 넘파이는 따로 설정하지 않아도 이를 활용한다**.
- 두 배열을 원소별로 곱하여 result에 담는 데, 벡터화 연산을 사용하여 보자. 이때 사칙연산자는 벡터화 연산자로 배열의 모든 원소에 대해 한꺼번에 적용된다.

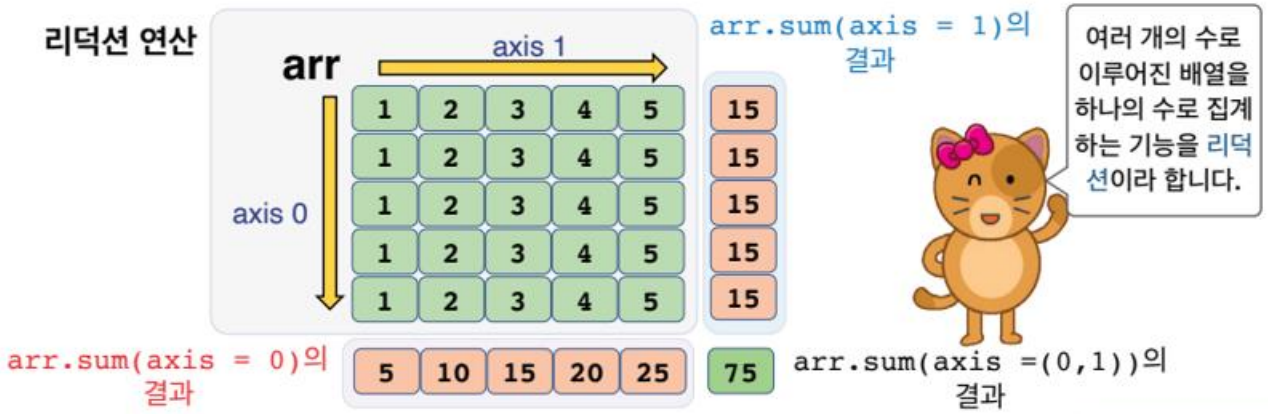
```
n_elem = 100000          # 원소의 개수
arr_a = np.random.rand(n_elem)
arr_b = np.random.rand(n_elem)
result = np.zeros(n_elem)
```

```
start = time.time()
for i in range(n_elem):                # 명시적인 작업 지시 : C 스타일
    result[i] = arr_a[i] * arr_b[i]
end = time.time()
print('원소별 접근 방식 소요시간: ', end - start)

start = time.time()
result = arr_a * arr_b
end = time.time()
print('벡터화 연산 수행 소요시간: ', end - start)
```

3.15 리덕션 : 배열을 더 강력하게 만드는 기능

- 넘파이의 다차원 배열의 **리덕션**reduction 기능은 여러 개의 수로 이루어진 배열을 하나의 수로 **집계**aggregation하는 것이다. 이 집계를 위한 함수로 sum(), mean(), min(), max() 같은 함수를 제공한다. 벡터화 연산이 적용되는 **리덕션 연산은 높은 성능을 보여 준다.**




```
arr = np.array([[1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
               ])
print(np.sum(arr, axis = 0))
print(np.sum(arr, axis = 1))
print(np.sum(arr, axis=(0, 1)))  # axis = (0, 1)은 생략 가능
```

```
[ 5 10 15 20 25]
[15 15 15 15 15]
75
```

```
sum_naive = 0
start = time.time()
for i in range(n):
    for j in range(n):
        sum_naive += arr[i,j]
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_naive, "\n계산 시간: ", end-start)
```

```
1000 × 1000 배열의 원소 합 500048.24990126066
계산 시간:  0.39148378372192383
```

```
start = time.time()
sum_reduction = np.sum(arr, axis = (0,1))
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_reduction, "\n계산 시간: ", end-start)
```

```
1000 × 1000 배열의 원소 합 500048.2499012628
계산 시간:  0.00128173828125
```

3.16 다차원 배열의 결합 concatenate, vstack, hstack

- **concatenate(a,b)** 함수는 a **배열의 마지막 원소 다음에 b** 배열의 원소를 차례로 넣어 배열을 결합하고 이를 반환. **vstack(a,b), hstack(a,c)**은 수직 방향과 수평 방향으로 다차원 배열을 결합한다.
- **vstack()은 배열과 리스트를 결합**, 다차원 배열 3개를 한꺼번에 결합. 반면 **hstack()은 axis 0의 차원값이 동일한 때에만 두 벡터를 수평축으로 결합**.

```
a = np.arange(10, 18).reshape(2,4)
a   # (2, 4) 형상의 다차원 배열로 10에서 17까지의 원소를 가짐
```

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17]])
```

```
b = np.arange(12).reshape(3,4)
b  # (3, 4) 형상의 다차원 배열로 0에서 11까지의 원소를 가짐
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
c = np.arange(8).reshape(2,4)
c   # (2, 4) 형상의 다차원 배열로 0에서 7까지의 원소를 가짐
```

```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

```
d = list(range(4))
d   # d는 1차원 리스트로 0에서 3까지의 원소를 가짐
```

```
[0, 1, 2, 3]
```

```
np.concatenate((a, b))   # a 배열의 마지막 원소 다음에 b 배열을 넣는다
```

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
np.vstack((a, b)) # a 배열과 b 배열을 수직 방향으로 결합
```

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
np.hstack((a, c))   # 다차원 배열 a, c를 수평으로 결합
```

```
array([[10, 11, 12, 13,  0,  1,  2,  3],
       [14, 15, 16, 17,  4,  5,  6,  7]])
```

```
# a 배열은 형상이 (2, 4), b 배열은 (3, 4)로 수평 결합이 안 됨
np.hstack((a, b)) # a 배열과 b 배열의 hstack 결합-오류
```

```
ValueError: all the input array dimensions for the concatenation axis must
match exactly, but along dimension 0, the array at index 0 has size 2 and
the array at index 1 has size 3
```

```
np.vstack((a, d)) # a 배열과 d 리스트의 결합
```

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3]])
```

```
np.vstack((a, b, d)) # 다차원 배열 3개를 결합
```

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [ 0,  1,  2,  3]])
```

3.17 배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

- r_은 큰 **괄호 []**안에 **연결할 원소를 넣거나 다차원 배열을 넣어서** 배열을 연결. 2차원 배열 이상은 **첫 번째 축인 axis=0 연결 방향이** 된다.
- 다음은 a, b의 각 원소끼리 묶어서 [1, 10], [2, 20], [3, 30]을 원소로 하는 다차원 배열을 만드는 방법인데, c_ 클래스, column_stack() 함수를 사용하여 묶어주는 방법도 있다.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[10, 20, 30], [40, 50, 60]])
```

```
np.r_[a, b]   # a, b 배열을 결합함 np.hstack([a, b])와 같다
```

```
array([ 1,  2,  3, 10, 20, 30])
```

```
np.r_[a, 4, 5, 6, b] # 중간에 개별 원소 삽입도 가능하다
```

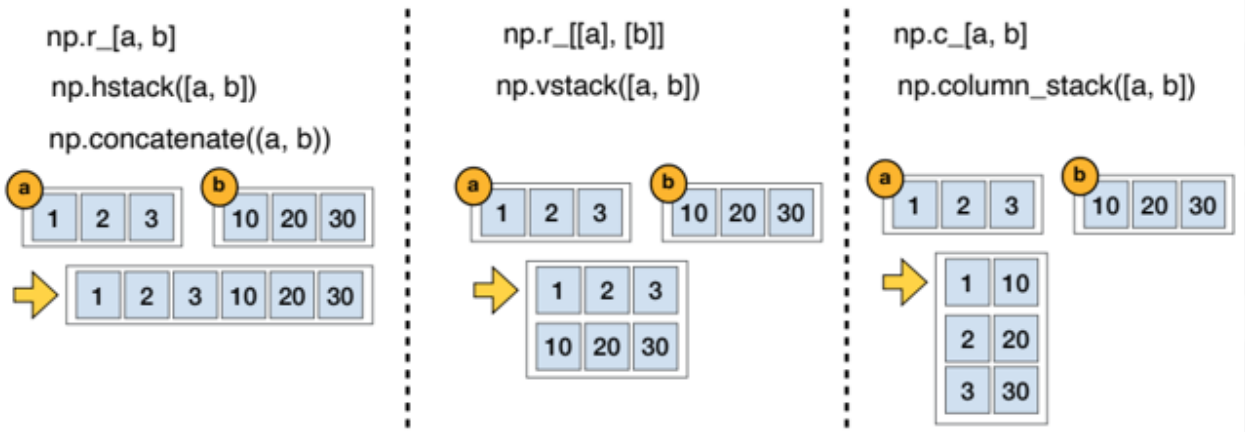
```
array([ 1,  2,  3,  4,  5,  6, 10, 20, 30])
```

```
np.r_[[0]*3, 5, 6]
```

```
array([0, 0, 0, 5, 6])
```

```
np.r_[c, d]  # 2차원 배열을 axis=0 방향으로 결합한다
```

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [10, 20, 30],
       [40, 50, 60]])
```




```
np.concatenate((a.reshape(-1, 1), b.reshape(-1, 1)), axis=1)
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

```
np.c_[a, b]
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

```
np.column_stack([a, b])
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

```
# c와 d의 전치행렬을 결합시킨다
np.concatenate((c.T, d.T), axis=1)
```

```
array([[ 1,  4, 10, 40],
       [ 2,  5, 20, 50],
       [ 3,  6, 30, 60]])
```

3.18 다차원 배열 결합 append()

- **append()** 함수는 **인자로 입력된 두 배열을 합하는**데 axis를 통해서 **축을 명시하지 않으면 1차원 배열로 평탄화하여 합침. 이때 axis = 0과 같이 축을 명시해 준다면 두 배열의 열의 개수가 같으면 수직(행기준)으로 합하게 되어 배열의 차원이 유지.**

```
a = np.array([1, 2, 3])
b = np.array([[4, 5, 6], [7, 8, 9]])
np.append(a, b)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.append([a], b, axis = 0)      # [a]를 통해 2차원 배열로 만들어야 함
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

3. 도전문제

도전문제 3.1 다차원 배열 생성하기

난이도 중

대각 성분이 모두 10인 4×4 크기의 다차원 배열을 eye() 함수와 브로드캐스팅 기능을 이용하여 생성하여라.

```
array([[10.,  0.,  0.,  0.],
       [ 0., 10.,  0.,  0.],
       [ 0.,  0., 10.,  0.],
       [ 0.,  0.,  0., 10.]])
```

도전문제 3.2 사인값 출력하기

난이도 중

sin0도에서 sin180도 사이의 값 11개를 다음과 같이 출력하여라. 이때 sin() 함수의 입력을 라디안을 사용하여라(힌트: 넘파이의 linspace() 함수를 사용해보자. 라디안으로 180도는 np.pi를 사용하면 된다).

```
[0.00000000e+00 3.09016994e-01 5.87785252e-01 8.09016994e-01
 9.51056516e-01 1.00000000e+00 9.51056516e-01 8.09016994e-01
 5.87785252e-01 3.09016994e-01 1.22464680e-16]
```

도전문제 3.5 다차원 행렬의 생성과 활용

난이도 중

1에서 9까지의 정수 값을 크기 순서대로 가지는 3×3 크기의 행렬 a를 arange()와 reshape()을 이용하여 생성하여라. 모든 성분의 값이 3인 3×3 크기의 행렬 b를 full() 함수를 사용하여 생성하여라. a와 b를 다음과 같이 출력하여라.

```
a = [[1 2 3]
     [4 5 6]
     [7 8 9]]
```

```
b = [[3 3 3]
     [3 3 3]
     [3 3 3]]
```

도전문제 3.6 다차원 배열을 나누어 보자

난이도 중

arange()를 사용하여 1에서 50까지의 원소를 가지는 다차원 배열을 만들자. 이 배열의 원소 50개를 랜덤하게 섞은 후 80%의 데이터는 train_data에 넣고 나머지 20%의 데이터는 test_data라는 배열에 넣어서 이 두 개의 배열을 반환하는 함수 train_test_split()을 만들자. 반환된 배열 값을 각각 출력하여라.